# System Workflow of a RAG-Based AI System

An in-depth look at Retrieval-Augmented Generation architecture for enhanced AI performance.

Presented by: Senior AI System Architect
For: Technical Evaluators & Externship Reviewers

Made with GAMMA

CHAPTER 1

# Step 1: User Query Submission

The initial interaction begins with the user articulating their information need, which then undergoes critical preparatory steps to ensure effective processing by the AI system.

### User Submits Natural Language Question

The user's input is typically in free-form text, expressing a specific query or task.

### Query Preprocessing (Optional Cleaning)

This stage involves tokenization, normalization, and removal of stop words to refine the input.

### Convert Query into Embedding

The preprocessed query is transformed into a dense vector representation using a trained embedding model.

### Prepare for Retrieval Search

The query embedding is now ready to be matched against the system's knowledge base for relevant information.

User Submits Query

Preprocessing

Generate Embedding

This diagram illustrates the initial phase where a user's natural language question is converted into a machine-understandable embedding for subsequent steps.

Made with GAMMA

CHAPTER 2

# Step 2: The Retriever Module

The Retriever is responsible for efficiently sifting through vast amounts of information to identify the most relevant data chunks that could answer the user's query.

### Semantic Similarity Search

The query embedding is compared with document embeddings to find semantically similar content.

### Vector Database Lookup

Efficient retrieval is performed using specialized vector databases optimized for high-dimensional nearest neighbor search.

### Top-K Relevant Chunks Retrieved

A predefined number (K) of the most similar document chunks are extracted for further processing.

### Hybrid Search (Optional)

Combines keyword-based search with semantic search to enhance retrieval accuracy and recall.

**Retriever: Vector Search**

**Top-K results**
Return most relevant docs

**Query embedding**
Convert query to vector

**Similarity search**
Compute nearest neighbors

**Vector database**
Store and index embeddings

The Retriever's role is to bridge the query with potential answers by finding and extracting the most relevant knowledge from the system's corpus.

Made with GAMMA

CHAPTER 3

# Step 3: The Reranker Module

The Reranker refines the initial set of retrieved documents, ensuring that only the most pertinent and high-quality information is passed to the Language Model.

- Re-evaluates Retrieved Chunks: Applies a more sophisticated scoring mechanism to refine relevance.
- Cross-Encoder Scoring: Uses a deeper neural network to understand the interaction between query and document chunks.
- Filters Less Relevant Results: Prunes low-scoring or redundant chunks to improve context quality.
- Selects Best Context: Delivers a refined, highly relevant subset of chunks to the LLM.

This diagram illustrates the Reranker's function in taking a broader set of retrieved documents and narrowing it down to the most precise and useful context.

Made with GAMMA

CHAPTER 4

# Step 4: LLM Processing with Context

With a refined context in hand, the Large Language Model generates an answer that is not only coherent but also factually accurate and directly grounded in the provided information.

1

### Context Injection into Prompt

The refined document chunks are concatenated with the user query, forming a comprehensive input for the LLM.

2

### Prompt Engineering Strategy

Carefully crafted prompts guide the LLM to utilize the context effectively and generate the desired output format.

3

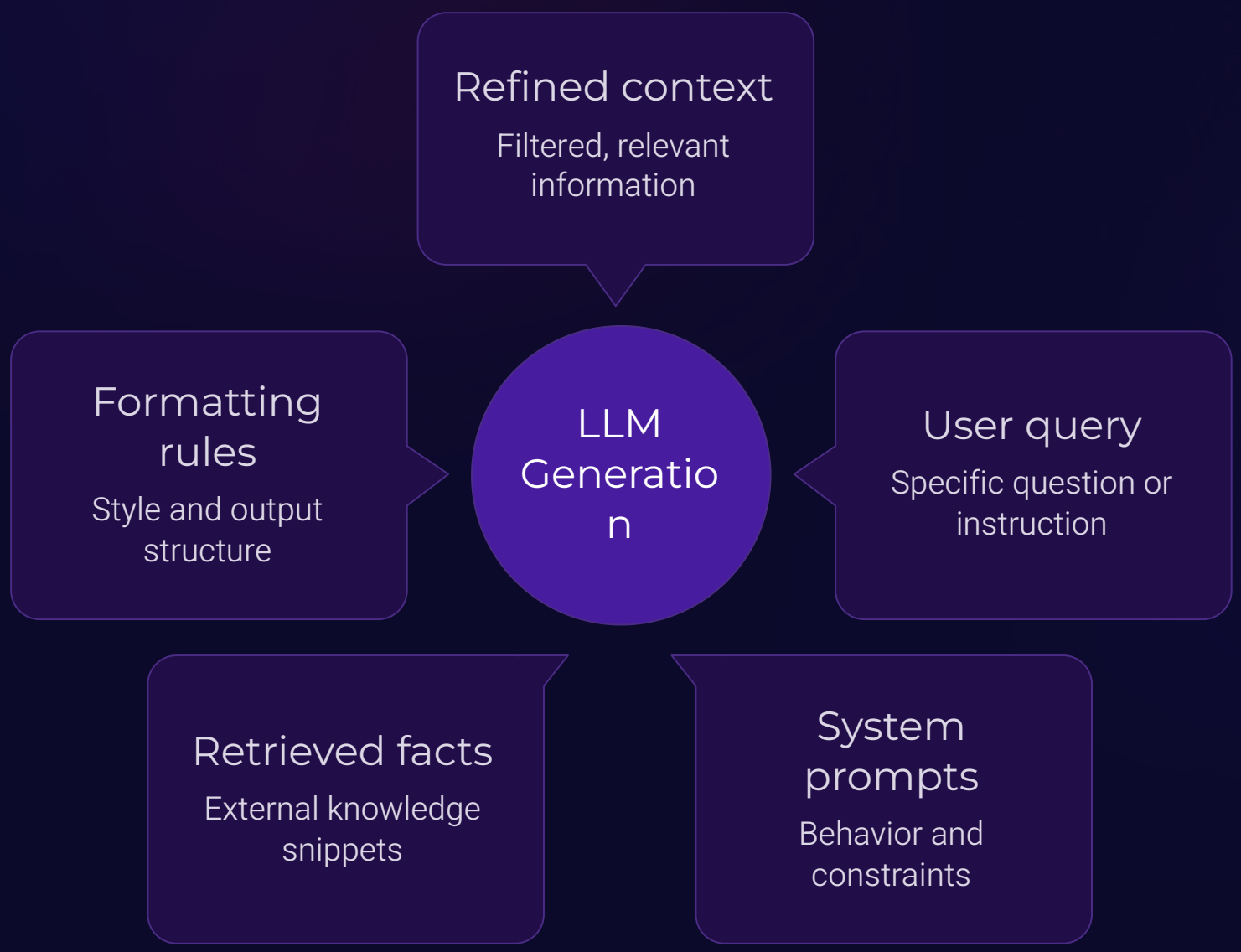
### Grounded Response Generation

The LLM generates an answer that is firmly rooted in the provided context, reducing speculative or incorrect information.

4

### Reduced Hallucination

By relying on retrieved facts, the system significantly mitigates the LLM's tendency to generate factually baseless content.




This visual represents how the combined refined context and user query power the LLM's generation capabilities.

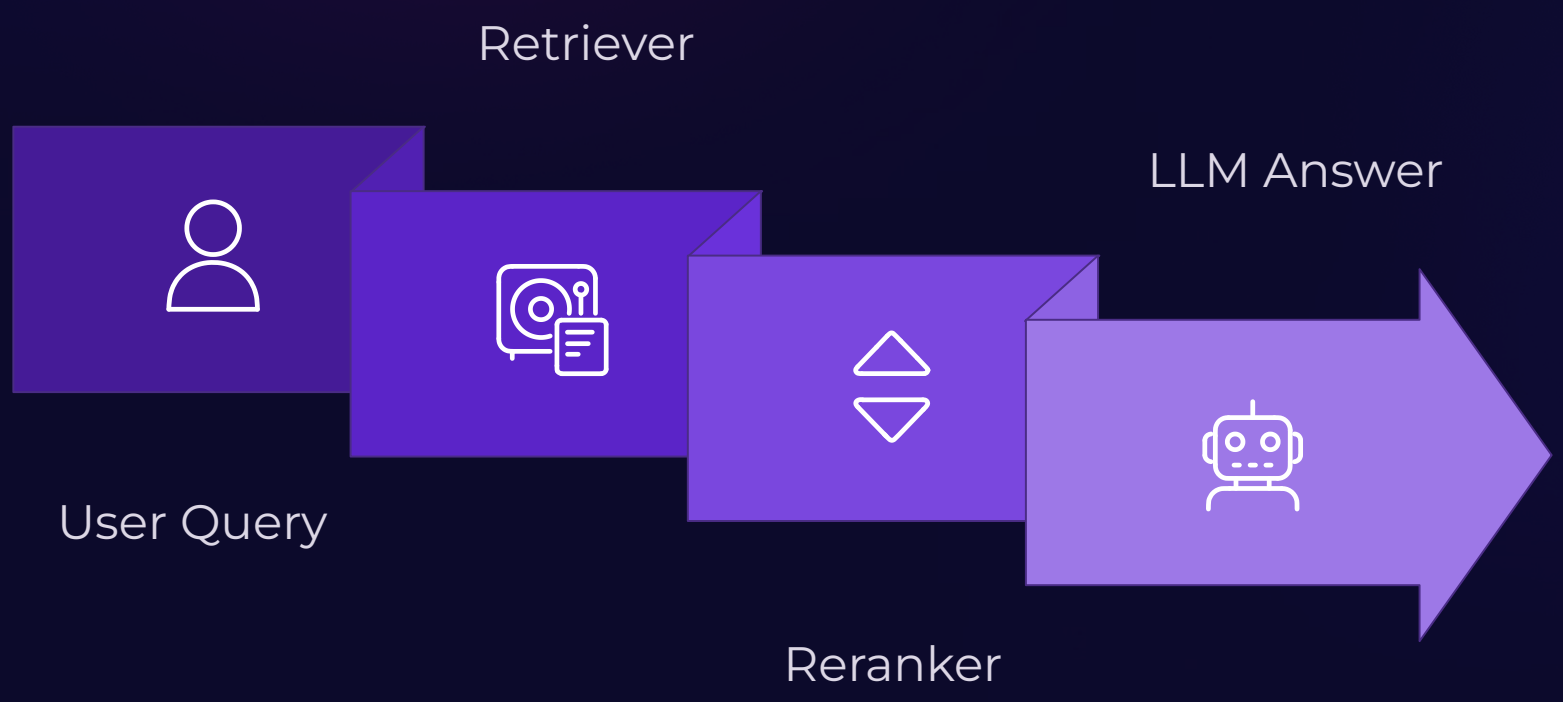
Made with GAMMA

CHAPTER 5

# Final Answer Generation & Optimization

The ultimate goal of the RAG system is to deliver a precise, coherent, and factually accurate answer to the user's initial query, completing an optimized end-to-end pipeline.

- Structured Response Output: Answers are formatted for clarity, often presenting information in bullet points or concise paragraphs.
- Context-Aware Answer: The generated response directly addresses the query using only the information from the retrieved context.
- Improved Factual Accuracy: Leveraging external knowledge drastically enhances the reliability of the LLM's output.
- End-to-End Optimized Pipeline: Each stage works synergistically to deliver superior performance and user experience.




This comprehensive diagram illustrates the entire RAG system workflow, from the initial user query to the final, context-aware answer.

Made with GAMMA